

UNIVERSITY OF GHANA
DEPARTMENT OF COMPUTER SCIENCE
DSCD610: REINFORCEMENT LEARNING



NAME: ETORNAM KOFI AGBETORNYO

STUDENT ID: 22425104

TOPIC: FROZEN LAKE USING Q-LEARNING

GITHUB NAME: Mister-Kayy

GITHUB REPOSITORY LINK: <https://github.com/Mister-Kayy/frozen-lake-qlearning>

1. Introduction

Reinforcement Learning

Reinforcement Learning (RL) is a branch of machine learning in which an agent learns to make decisions by interacting with an environment. At each time step, the agent observes a state, selects an action and receives a reward along with the next state.

Frozen Lake Problem

Frozen Lake is a grid reinforcement learning environment where an agent must navigate from a start position to a goal while avoiding holes. Each cell in the grid is represented as:

Symbol Meaning

S	Start state
F	Frozen (safe) state
H	Hole (terminal failure)
G	Goal (terminal success)

The objective is to learn an optimal policy that maximizes the probability of reaching the goal while avoiding holes. The environment used in this project is an 8×8 grid:

```
SFFFFFFF
FFFFFFF
FFFHFFFF
FFFHFFFF
FFFHFFFF
FHHFFFFH
FHFFHFHF
FFFHFFFG
```

2. Environment Design

State Representation

The environment uses a discrete state representation based on grid coordinates mapped to a single index which produces 64 unique states for the 8×8 grid, enabling direct indexing into a Q-table.

Action Representation

Reward Structure

Action Value Movement			Event	Reward Terminal	
Left	0	(0, -1)	Move to frozen tile (F)	-0.01	No
Down	1	(1, 0)	Fall into hole (H)	-1.0	Yes
Right	2	(0, 1)	Reach goal (G)	+10.0	Yes
Up	3	(-1, 0)			

A small negative step penalty encourages shorter paths and discourages random wandering.

Episodes are also limited to 200 steps to prevent infinite loops.

3. Q-Learning Algorithm

Q-Learning is a model-free, off-policy reinforcement learning algorithm that learns an action-value function:

$$Q(s, a)$$

This function estimates the expected future reward of taking action a in state s . The agent does not require knowledge of environment dynamics and learns purely from experience.

Update Rule

The Q-value is updated using:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

Where:

- α = learning rate
- γ = discount factor
- r = reward
- s' = next state

Exploration Strategy (ϵ -greedy)

The agent uses an ϵ -greedy policy which chooses a random action (exploration) and a probability of $1 - \epsilon$ which chooses the best known action (exploitation) . The ϵ starts at 1.0 and decays exponentially to 0.01 over training, ensuring sufficient exploration early in training before gradually shifting toward exploitation.

4. Training Methodology

Hyperparameters

Parameter	Value	Description
α (learning rate)	0.1	Controls Q-value updates
γ (discount factor)	0.99	Future reward importance
ϵ start	1.0	Initial exploration
ϵ min	0.01	Minimum exploration
ϵ decay	computed	Exponential decay
Max steps	200	Episode limit

These parameters were selected through controlled experimentation, adjusting one parameter at a time to evaluate its effect on performance.

Training Setup

The agent was trained for 5,000 episodes, allowing sufficient exploration and convergence.

5. Experimental Results

Training Performance

- Final training success rate: **84.7%** (including early exploration)
- Final rolling success rate (last 500 episodes): **99%**
- Greedy evaluation success rate (100 episodes): **100%**
- Average greedy reward: **9.87**

The improvement over time demonstrates convergence from exploration to an optimal policy.

6. Learned Policy

The final policy derived from the Q-table is:

```
→ ↓ ↓ ↓ ↓ ↓ → ↓
→ → → → ↓ ↓ → ↓
↑ ↑ ↑ H ↓ ↓ ↓ ↓
↑ ↑ ← H → → → ↓
↓ ↑ ↑ H → → → ↓
↓ H H → → ↑ H ↓
↓ H → ↑ H ↑ H ↓
→ → ↑ H ← ← → G
```

The learned policy successfully guides the agent from start to goal while avoiding holes. The optimal path discovered is near-minimal in length, indicating efficient convergence.

7. Challenges Encountered

Several challenges were encountered during implementation. These challenges are;

- The sparse reward structure only received upon reaching terminal states which slowed early learning.
- A second challenge was balancing exploration and exploitation. High initial exploration caused randomness, while rapid decay reduced learning diversity.
- Another issue was uneven state visitation, where states outside the optimal path were rarely updated, leading to less reliable Q-values in those regions.

Despite these challenges, the agent successfully converged to an optimal policy.

8. Conclusion

This project implemented a Q-Learning agent from scratch to solve the Frozen Lake environment. The agent successfully learned an optimal policy through iterative interaction and application of the Bellman update equation. Over time, exploration decreased and the agent shifted toward exploitation of learned knowledge. Final evaluation results demonstrated a 100% success rate under greedy policy execution, confirming convergence to an optimal solution.